MENA Prediction Market
Complete Project Audit & Go-Live Plan

Full A-Z audit of local code, GitHub, CI/CD, database, every flow,
all bugs, env vars, observability gaps, and what's needed to go live.

Date: March 29, 2026 | Branch: staging | Commit: d7b4db0

Item	Value
GitHub	S-oftware-F-actory/prediction-market (private)
Branches	main, staging, demo - all synced at d7b4db0
Infra Decision	Snapshot replica: production DB nightly-copied to demo DB
Default Branch	main
Commits	19 total
Source Files	179 TypeScript/TSX
SQL Migrations	68 files
Working Tree	Clean

PART 2: COMPLETE INVENTORY - WHAT'S BUILT

A. Trading Engine - COMPLETE

- LMSR AMM: lmsr_cost, lmsr_price, lmsr_shares_for_cost (numerically stable log-sum-exp)
- execute_trade - buy/sell with explicit fees, cash-out premium, deterministic locking
- get_amm_price, get_cash_out_value - real-time pricing + sell preview
- initialize_amm - per-market AMM with configurable liquidity parameter (b=1000)
- Both-side trading (YES + NO on same market)

B. Market Lifecycle - COMPLETE

- resolve_market - pays winners (99c/share), settles commissions, records revenue, auto-voids if no winners
- void_market - full refund + commission clawback
- lock_market - close to new trades
- Statuses: draft -> open -> closed -> resolved/voided

C. Financial System - COMPLETE

- Append-only transactions ledger (source of truth, balance_usd is cache)
- process_deposit - idempotent on 3pay/Whish ref
- process_withdrawal - checks wagering req + 24hr delay
- claim_deposit_bonus - $5 for non-referred users (first deposit >= $20)
- reconcile_balances - cron detects ledger/cache mismatches
- All fees from fee_config table - ZERO hardcoding

D. Commission Engine - COMPLETE

- 3-deep multi-level referral commissions on 0.5% explicit trading fee
- 4 agent tiers (L1-L4) with configurable rates from fee_config
- settle_commissions, pay_trade_commissions, settle_resolution_commissions
- update_agent_level - ratchet based on referral count (10->L2, 25->L3, 50->L4)
- handle_referral_signup - chain building (max 3 ancestors)

E. Database - COMPLETE (39 tables)

- Core: users, markets, trades, positions, transactions, deposits, withdrawals
- AMM: amm_state, price_alerts
- Social: market_comments, comment_likes, comment_replies
- Copy trading: copy_settings, leader_stats (schema only)
- System: system_logs, fee_config, help_center, user_locations
- RLS on ALL tables, auth.uid() enforcement, SELECT FOR UPDATE on mutations

F. All 25+ RPC Functions - COMPLETE

Function	Status
execute_trade	DONE - v2 (latest, with NGR)
resolve_market	DONE - v2 (latest)
void_market / lock_market	DONE
process_deposit	DONE (idempotent)
process_withdrawal	DONE

Function	Status
claim_deposit_bonus	DONE
settle_commissions	DONE
pay_trade_commissions	DONE
settle_resolution_commissions	DONE
record_revenue	DONE - v2
update_agent_level	DONE - v2
handle_referral_signup	DONE
initialize_amm	DONE
get_amm_price / get_cash_out_value	DONE
lmsr_cost / lmsr_price	DONE (IMMUTABLE)
reconcile_balances	DONE
withdrawal_approve/reject	DONE
toggle_user_freeze	DONE
dynamic_max_bet / dead_market_check	DONE
log_system_event	DONE

G. Frontend - User App (35 routes, 50+ components)

Page	Status	Notes
/ (Home)	COMPLETE	Hero market, secondary grid, sidebar
/market/[id]	COMPLETE	Price chart, trade panel, comments, related
/portfolio	COMPLETE	P&L stats, position cards, cash-out
/wallet	COMPLETE	Balance, deposit/withdraw, transactions
/wallet/deposit	COMPLETE	3pay/Whish, presets, bonus eligibility
/wallet/withdraw	COMPLETE	Fee calc, address, wagering check
/profile	COMPLETE	P&L chart, positions/activity tabs
/settings	COMPLETE	Profile, Account, Notifications
/notifications	COMPLETE	Read/unread, bilingual
/help	COMPLETE	Collections, articles, search, TOC
/login	COMPLETE	Phone OTP -> profile completion
/referral	PARTIAL	Basic page, agent dashboard not built
/leaderboard	STUB	Directory exists, no implementation

H. Frontend - Admin Dashboard (~85% Complete)

Page	Status	Notes
/admin	COMPLETE	Revenue, deposits, AMM health, risk
/admin/markets	COMPLETE	List, create, volume/trades
/admin/markets/create	COMPLETE	Form with icon picker
/admin/markets/[id]	PARTIAL	
/admin/markets/[id]/resolve	COMPLETE	Two-step resolution
/admin/users	COMPLETE	List with stats
/admin/users/[id]	PARTIAL	
/admin/withdrawals	COMPLETE	Approval queue
/admin/fees	COMPLETE	Platform + commission editor
/admin/finance	COMPLETE	Deposits/withdrawals/ledger tabs
/admin/agents	PARTIAL	
/admin/amm	PARTIAL	
/admin/alerts	COMPLETE	4-card dashboard
/admin/logs	COMPLETE	Real-time log viewer
/admin/help/*	COMPLETE	CRUD for articles/collections

I. UI Components - 50+ Total

- **14 shadcn/ui:** Badge, Button, Card, Confetti, Dialog, Dropdown, Input, Odometer, Sheet, Skeleton, Sonner, SwipeToConfirm, Tabs, ToggleSwitch

- **14 Market:** trade-panel, price-chart, market-comments, hero-market-card, cash-out-sheet, etc.
- **8 Wallet:** balance-display, deposit-modal, withdraw-modal, transaction-list, etc.
- **7 Auth:** auth-guard, auth-steps, login-modal, otp-input, phone-input, etc.
- **17 Admin:** data-table, stat-card, fee-config-editor, markets-table, users-table, etc.
- **7 Layout:** top-nav, bottom-nav, balance-chip, profile-dropdown, notification-dropdown

K. CI/CD - COMPLETE

- 6 workflows: ci.yml, deploy-staging, deploy-demo, deploy-production, simulate, migration-check
- Three environments: staging / demo / production
- PR checks: lint -> typecheck -> build -> test
- Production: manual approval -> migrations -> health check

L. Observability - GOOD (with gaps)

Layer	Tool	Status
Client errors	Sentry (100% capture)	DONE
Server errors	Sentry + structured logger	DONE
DB errors	system_logs table + log_system_event() RPC	DONE
Session replay	Sentry (100% on errors, 0% normal)	DONE
Performance	Sentry traces (10% sampling)	DONE
Health check	/api/health (DB connectivity)	DONE
Cron monitoring	/api/cron/check-errors (every 10 min)	DONE
Balance reconciliation	reconcile_balances() via cron	DONE
Admin logs UI	/admin/logs (real-time viewer)	DONE
Admin alerts UI	/admin/alerts (lopsided, mismatches)	DONE
Real-time alerting	No email/Slack/PagerDuty integration	MISSING
Product analytics	No Vercel Analytics, PostHog, etc.	MISSING

M. Tests - 38 Cases, 6 Suites

Suite	Cases	Status
place-bet	11	~5 are placeholder stubs
resolve-market	10	~8 are placeholder stubs
payout-invariants	8	Real tests
deposit-withdrawal	7	~1 placeholder
race-conditions	3	Real tests
commission	6	Real tests

Root cause of stubs: auth.uid() mocking not implemented for user-facing RPCs.

PART 3: ALL KNOWN BUGS & ISSUES

Critical

#	Issue	File(s)	Impact
1	~20+ as any type assertions in admin pages	admin/*.tsx	Type errors hidden at runtime
2	Realtime subscriptions use as any	price-chart, comments	Could crash on schema change

High

#	Issue	File(s)	Impact
3	~15 test cases are expect(true).toBe(true) stubs	tests/db/*.test.ts	Critical paths untested
4	Hardcoded placeholder images in trending sidebar	trending-sidebar.tsx	Non-functional feature
5	@ts-expect-error on Sentry instrumentation	instrumentation.ts	Could break on upgrade
6	ESLint config broken (circular reference)	eslint.config.mjs	npm run lint fails

Medium

#	Issue	File(s)	Impact
7	RPC responses cast without validation	hooks/*.ts	Silent data corruption risk
8	Notification operations cast to any	notification-dropdown.tsx	Could break on API change
9	Admin data table row key as any	admin-data-table.tsx	Rendering issues

Low

#	Issue	File(s)	Impact
10	Console.log in production (structured JSON)	logger.ts	Verbose logs
11	Legacy comments reference old model	constants.ts	Code clarity
12	dangerouslySetInnerHTML for theme script	layout.tsx	Safe (hardcoded)

PART 4: ALL ENVIRONMENT VARIABLES

Local Development (.env.local)

Supabase (https://supabase.com/dashboard)

Variable	Status	Where Used
NEXT_PUBLIC_SUPABASE_URL	FILLED	client.ts, server.ts, middleware.ts, API routes
NEXT_PUBLIC_SUPABASE_ANON_KEY	FILLED	client.ts, server.ts, middleware.ts
SUPABASE_SERVICE_ROLE_KEY	FILLED	webhooks, cron, health, scripts, tests

Sentry (https://sentry.io)

Variable	Status	Where Used
NEXT_PUBLIC_SENTRY_DSN	FILLED	sentry.client.config.ts
SENTRY_DSN	FILLED	sentry.server.config.ts, sentry.edge.config.ts
SENTRY_AUTH_TOKEN	FILLED	next.config.ts (source map upload)
SENTRY_ORG	FILLED	next.config.ts
SENTRY_PROJECT	FILLED	next.config.ts

Payment Providers

Variable	Status	Where Used	Setup
THREEPAY_WEBHOOK_SECRET	EMPTY	/api/webhook/3pay	Contact 3pay
WHISH_WEBHOOK_SECRET	EMPTY	/api/webhook/whish	Contact Whish
WHISH_API_KEY	EMPTY	Future use	Contact Whish

Internal

Variable	Status	Where Used
CRON_SECRET	FILLED	/api/cron/check-errors

GitHub Actions Secrets (per environment)

Secret	Env	Status	Setup
SUPABASE_ACCESS_TOKEN	All	Needs setup	supabase.com/dashboard/account/tokens
SUPABASE_PROJECT_REF	All	Per-env	Project Settings > General > Project ID
NEXT_PUBLIC_SUPABASE_URL	All	Per-env	Project Settings > API
SUPABASE_SERVICE_ROLE_KEY	All	Per-env	Project Settings > API
SUPABASE_DB_URL	Demo	Needs setup	Project Settings > Database
SUPABASE_DB_PASSWORD	Demo	Needs setup	Project Settings > Database
PRODUCTION_URL	Prod	Needs setup	Your production domain

Accounts to Create/Configure

Service	URL	What You Need	Status
Supabase	supabase.com	3 projects (staging/demo/prod)	Staging exists, need demo+prod
Sentry	sentry.io	Org + project	Done
Vercel	vercel.com	Project linked to GitHub	Done
3pay	Contact 3pay	Merchant account + webhook secret	Not started
Whish	Contact Whish	Dev account + API key + webhook	Not started
GitHub	github.com	Environments + secrets	Partially done
Domain	Your choice	Production domain	Not started

PART 5: WHAT'S NEEDED TO GO LIVE

Tier 1 - BLOCKERS (Must Complete)

1. Payment Provider Integration

- [ ] Contact 3pay -> get merchant account -> get webhook secret
- [ ] Contact Whish -> get developer account -> get webhook + API key
- [ ] Set THREEPAY_WEBHOOK_SECRET and WHISH_WEBHOOK_SECRET in .env.local + Vercel
- [ ] Test deposit flow end-to-end with real money (small amounts)
- [ ] Verify webhook HMAC validation works with real signatures

2. Supabase Projects (Production + Demo Snapshot Replica)

Production project:

- [ ] Create production Supabase project
- [ ] Apply all 68 migrations to production
- [ ] Seed fee_config table with production rates
- [ ] Create admin user(s)
- [ ] Verify RLS policies work on production
- [ ] Set production env vars in Vercel + GitHub

Demo project (snapshot replica):

- [ ] Create demo Supabase project (same region as production)
- [ ] Set up nightly snapshot: pg_dump from prod -> pg_restore to demo
- [ ] Optionally run demo_helpers.sql post-restore
- [ ] Set demo env vars in Vercel + GitHub
- [ ] Verify simulation engine works against demo DB

3. Production Domain

- [ ] Register/configure domain
- [ ] Point DNS to Vercel
- [ ] SSL auto-provisioned by Vercel
- [ ] Set PRODUCTION_URL in GitHub secrets

4. GitHub Environments & Secrets

- [ ] Create staging, demo, production environments in GitHub Settings
- [ ] Add all required secrets per environment
- [ ] Enable "Required reviewers" on production environment
- [ ] Test CI/CD pipeline end-to-end

5. ESLint Fix

- [ ] Fix circular config in eslint.config.mjs
- [ ] Verify npm run lint passes
- [ ] Verify CI lint step passes

Tier 2 - HIGH PRIORITY (Before Launch)

6. Real-Time Alerting

- [ ] Choose channel: email (Resend/SendGrid) or Slack webhook
- [ ] Add alert trigger in /api/cron/check-errors
- [ ] Alert on: critical errors, balance mismatches, lopsided markets
- [ ] Test alert delivery

7. Product Analytics

- [ ] Enable Vercel Analytics (one-line add in layout.tsx)
- [ ] Enable Vercel Speed Insights
- [ ] Consider PostHog for product analytics

8. Fix Placeholder Tests

- [ ] Implement auth.uid() mock for Supabase test environment
- [ ] Replace ~15 expect(true).toBe(true) stubs with real assertions
- [ ] Run full test suite, verify all pass

9. Type Safety Cleanup

- [ ] Replace as any in admin pages with proper types
- [ ] Add runtime validation on RPC responses
- [ ] Remove @ts-expect-error from instrumentation.ts

10. Rate Limiting

- [ ] Add rate limiting middleware for API routes
- [ ] Consider Vercel WAF or Cloudflare for DDoS protection

Tier 3 - MEDIUM PRIORITY (Post-Launch)

- [ ] Agent/Referral Dashboard - build out /referral/agent with real data
- [ ] Leaderboard Page - implement with DB materialized view
- [ ] Sentry Alert Rules - email on new issues, error spikes
- [ ] Operational Runbook documentation

Tier 4 - POST-LAUNCH

- [ ] Copy trading UI (schema exists, no frontend)
- [ ] Push notifications
- [ ] Advanced charting
- [ ] A/B testing framework
- [ ] Canary deployments

PART 6: OPERATIONS PLAN

A. Daily Operations

Morning Checklist

1. Check Sentry dashboard for new errors (sentry.io)
2. Check /admin/alerts for balance mismatches, lopsided markets
3. Check /admin/logs for unacknowledged critical/error logs
4. Review pending withdrawals at /admin/withdrawals
5. Check Vercel deployment status (vercel.com)

Market Operations

- **Create market:** /admin/markets/create -> set question, sides, close date, icon
- **Initialize AMM:** Happens automatically on market creation (b=1000 default)
- **Lock market:** /admin/markets/[id] -> Lock action (stops new trades)
- **Resolve market:** /admin/markets/[id]/resolve -> two-step confirmation -> selects winner
- **Void market:** /admin/markets/[id] -> Void action -> refunds all + claws back commissions

Financial Operations

- **Approve withdrawal:** /admin/withdrawals -> review -> approve/reject
- **Deposits:** Automatic via webhooks (3pay/Whish) - no manual action
- **Fee changes:** /admin/fees -> edit rates (takes effect immediately)
- **Balance reconciliation:** Automatic every 10 min via cron

B. Incident Response

Error Detected (Sentry / Admin Alerts)

1. Check Sentry for error details + session replay
2. Check /admin/logs for DB-level error context
3. If balance mismatch: run reconcile_balances() manually, investigate
4. If trade error: check system_logs for specific trade failure context (JSONB)
5. If critical: consider locking affected market(s) while investigating

Market Issue

1. Lock the market immediately (lock_market)
2. Investigate (check trades, positions, AMM state)
3. If unresolvable: void the market (full refund)
4. If resolvable: unlock and continue

Payment Issue

1. Check webhook logs in Vercel (Functions tab)
2. Verify HMAC signature validation
3. Check deposit/withdrawal status in /admin/finance
4. If stuck deposit: check deposits table for duplicate threepay_ref
5. If stuck withdrawal: check withdrawals table status

C. Deployment Workflow

Feature Development

1. Create feature branch from staging
2. Develop + test locally

3. Push -> PR to staging
4. CI runs: lint -> typecheck -> build -> test
5. Review + merge to staging
6. Auto-deploys to staging preview
7. Test on staging
8. PR from staging -> main
9. Manual approval in GitHub
10. Migrations applied to production
11. Health check passes
12. Production live

Hotfix

1. Create hotfix branch from main
2. Fix + test
3. PR to main (skip staging for urgency)
4. Manual approval + deploy
5. Cherry-pick back to staging

PART 7: TECHNICAL WORKFLOW

Adding a New Feature

Step 1: Plan

- Read relevant docs (CLAUDE.md, DESIGN.md, docs/)
- Identify affected: pages, components, RPC functions, migrations, tests

Step 2: Backend (if needed)

- Write migration in supabase/migrations/NNN_description.sql
- Add tables, RPC functions, RLS policies
- Follow patterns: auth.uid() for user RPCs, SELECT FOR UPDATE, fees from fee_config
- Test locally with supabase db push

Step 3: Frontend

- Add/edit pages in src/app/(app)/ or src/app/admin/
- Use existing shadcn/ui components from src/components/ui/
- Follow DESIGN.md (colors, fonts, spacing, dark/light)
- Use supabase.rpc() for mutations, supabase.from().select() for reads
- Add loading states (Skeleton), error states, empty states

Step 4: Test

- Add DB test cases in src/tests/db/ if touching RPC functions
- Run npm test to verify
- Run npm run build to verify no TS errors

Step 5: Ship

- Push to staging branch
- CI validates
- Test on staging preview
- PR to main when ready

Fixing a Bug

Step 1: Investigate

- Check Sentry for error details + replay
- Check /admin/logs for DB context
- Read the affected code

Step 2: Root Cause

- Identify the exact line/function causing the issue
- Check if it's a frontend, backend, or data issue

Step 3: Fix

- Fix the code
- If DB fix: write a migration (never modify existing migrations)

Step 4: Verify

- Run tests: npm test
- Build: npm run build
- Test the specific flow that was broken

Step 5: Deploy

- Same deployment workflow as features

Key Files to Know

Purpose	File
Supabase client (browser)	src/lib/supabase/client.ts
Supabase client (server)	src/lib/supabase/server.ts
Supabase middleware (auth)	src/lib/supabase/middleware.ts
Auth guards	src/lib/auth/guards.ts
Logger	src/lib/logger.ts
Constants	src/lib/constants.ts
Types	src/types/
Hooks	src/hooks/
Trade panel	src/components/market/trade-panel.tsx
Price chart	src/components/market/price-chart.tsx
All migrations	supabase/migrations/
Test helpers	src/tests/db/helpers.ts
CI pipeline	.github/workflows/ci.yml
Deploy workflows	.github/workflows/deploy-*.yml

PART 8: SCORECARD SUMMARY

Area	Status	Score
Trading Engine (LMSR AMM)	Complete	10/10
Market Lifecycle	Complete	10/10
Financial System	Complete	10/10
Commission Engine	Complete	10/10
Database + Security	Complete	10/10
Frontend - User App	Complete	9/10
Frontend - Admin	~85%	8/10
Design System	Complete	9/10
CI/CD Pipeline	Complete	9/10
Observability (error capture)	Good	8/10
Observability (alerting)	Missing	3/10
Observability (analytics)	Missing	0/10
Tests (DB logic)	~60% real	6/10
Tests (Frontend/E2E)	None	0/10
Payment Integration	Stub	2/10
Production Infrastructure	Partial	5/10
Operational Readiness	Partial	5/10

Overall: The code is ~90% built and production-quality. The remaining ~10% is operational: payment integration, production infra, alerting, and test coverage.